# TESTING AGILE

La idea central:

- **Testing tradicional:** el testing suele concentrarse en encontrar defectos mediante pruebas realizadas después de las actividades de desarrollo.
- **Testing ágil:** el testing se integra al desarrollo y busca prevenir defectos, detectarlos tempranamente y obtener feedback rápido.

La diferencia está en **cómo se integra el testing al proceso y cuál es el enfoque predominante.**

Temas:




## MANIFIESTO DEL TESTING

El Manifiesto de Testing Ágil surge de la comunidad de testing ágil, tomando como referencia el Manifiesto Ágil, pero enfocado específicamente en cómo se conciben y realizan las pruebas de software.

La idea fundamental es que, si cambia la forma de desarrollar software, también cambia la manera de entender el testing: deja de ser una etapa aislada y pasa a formar parte del trabajo cotidiano de todo el equipo.

## Los 5 valores del testing ágil

Estos valores expresan la manera de pensar el testing ágil.




1. Testing es una **actividad continua**, no una fase.

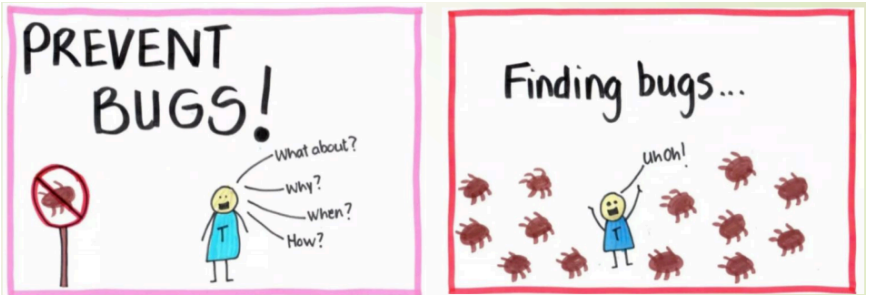
Agile Tradicional




En el tablero de la izquierda las tareas de desarrollo (D) y testing (T) están mezcladas, porque se hacen en paralelo. En el de la derecha hay una columna DEV y otra TEST: primero se desarrolla y **recién después** se prueba. Por eso el testing es "una fase".

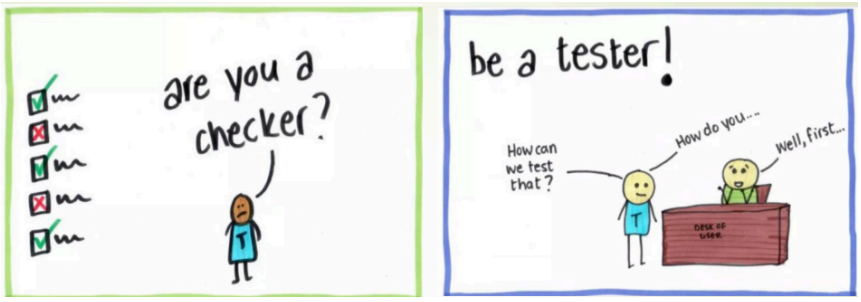
2. Evitar que los defectos se introduzcan en el software, en lugar de depender de detectarlos después.

Agile Tradicional




3. Evaluar si el software responde a las necesidades del negocio y del usuario, no solamente si cada función responde técnicamente.

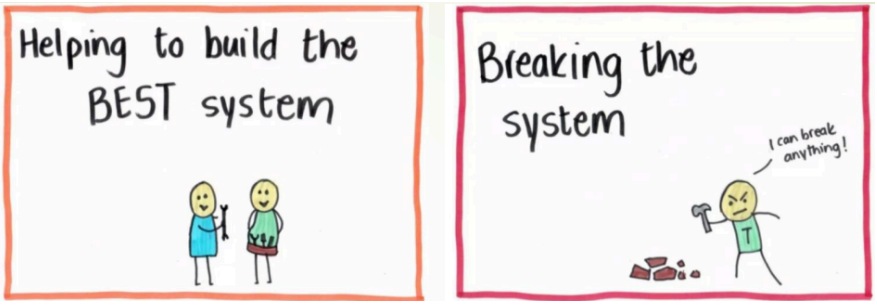
Tradicional Agile




"Be a tester!": pregunta "¿cómo podemos probar eso?" sentado con el usuario. "Are you a checker?": solo verifica que el sistema haga lo que dice la lista.

4. El tester colabora con el equipo para construir un producto de calidad, no es el que "rompe lo que hizo el desarrollador".

Agile | Tradicional




"Helping to build the BEST system": dos personas colaborando. "Breaking the system": el tester con un martillo, "puedo romper cualquier cosa".

5. La calidad y el testing involucran a todo el equipo, no solamente a quien tiene el rol de tester.




Si el tester es el único responsable, queda como el "portero" que decide si el software sale o no. En ágil la calidad es de todo el equipo, todos dicen "¡Listos para entregar!".

El valor más importante es la **prevención de defectos**, porque representa el cambio de mentalidad respecto del testing tradicional.

## 9 Principios para Testing en Proyectos Ágiles

**1. El testing se mueve hacia adelante en el proyecto:** Las pruebas empiezan antes y acompañan al desarrollo, en lugar de concentrarse al final.

**2. Testing no es una fase:** Es una actividad continua integrada al ciclo de vida del software.

**3. Todos hacen testing:** La responsabilidad de la calidad es compartida entre testers, desarrolladores y demás integrantes del equipo.

**4. Reducir la latencia del feedback:** Obtener información sobre errores o problemas lo antes posible, para corregirlos rápidamente.

**5. Las pruebas representan expectativas:** Las pruebas expresan qué resultados se esperan del software según los requisitos y las necesidades del negocio.

**6. Mantener el código limpio y corregir los defectos rápido:** La calidad debe mantenerse durante el desarrollo, evitando acumular problemas que después sean más costosos de solucionar.

**7. Reducir la sobrecarga de documentación de las pruebas:** Evitar documentación innecesaria o excesiva y priorizar la información útil para el equipo.

**8. Las pruebas son parte del "Done":** Una funcionalidad no se considera terminada solamente porque se programó: debe cumplir los criterios de calidad y las pruebas correspondientes.

**9. De probar al final a ser conducido por pruebas:** Las pruebas orientan el desarrollo, en lugar de limitarse a verificar el producto una vez implementado.

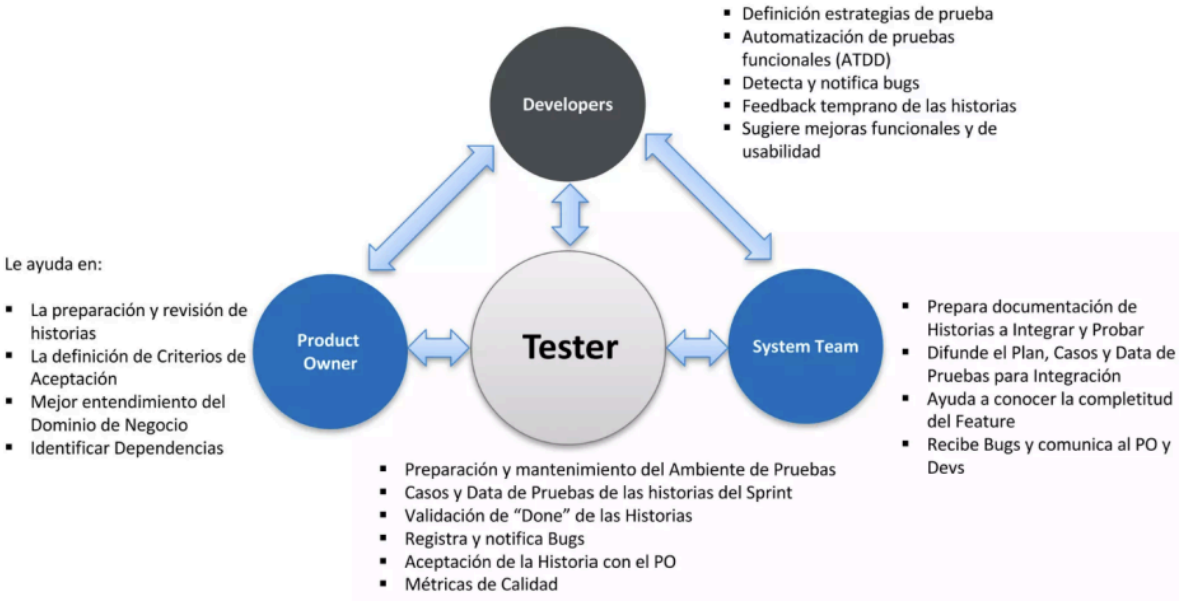
Estos principios son conceptuales: expresan hacia dónde se quiere orientar el trabajo del equipo. Para llevarlos a la práctica, se utilizan técnicas y herramientas concretas.

## PRÁCTICAS CONCRETAS DEL TESTING ÁGIL

Se definen 6 Prácticas Concretas. A diferencia de los principios, estos indican acciones específicas que **ayudan a aplicar el enfoque ágil.**
Es decir, aplicarlas ayuda a hacer ágil, pero no garantiza serlo.

1. **Pruebas de unidad e integración automatizadas:** Comprobar automáticamente componentes individuales y sus interacciones. Se ejecutan solas, rápido y muchas veces.
2. **TDD (Test Driven Development):** Desarrollar guiándose por pruebas que se escriben antes del código de producción.
3. **Pruebas de Regresión a nivel de sistema automatizadas:** Volver a comprobar funcionalidades existentes para verificar que los cambios no hayan introducido fallas. Automatizarla permite repetirla en cada cambio.
4. **ATDD (Desarrollo conducido por Pruebas de aceptación):** Guiar el desarrollo a partir de pruebas de aceptación que expresan las expectativas del negocio.
5. **Pruebas exploratorias:** Pruebas manuales donde el tester investiga el sistema con creatividad, aprendiendo y diseñando pruebas a la vez, para encontrar lo que las pruebas automáticas no cubren.
6. **Control de versión de las pruebas con el código:** Las pruebas viven en el mismo repositorio que el código, así cada versión del código tiene sus pruebas correspondientes.

## EL ROL DEL TESTER EN EQUIPOS ÁGILES




En el enfoque ágil, el tester tiene un rol más colaborativo que el que suele tener en un proceso tradicional. Es decir, el rol **cambia mucho**: en lugar de ser un rol estático y aislado, tiene un **vínculo fluido con todos los demás**.

El tester es el centro y sus tareas incluyen:

- Colaborar con el Product Owner (PO) para aclarar historias de usuario y criterios de aceptación.
- Validar que las historias de usuario satisfagan las necesidades del negocio.
- Trabajar junto con desarrolladores y otros integrantes del equipo para prevenir defectos.
- Diseñar y ejecutar pruebas, analizar resultados y comunicar problemas.
- Ayudar a que las expectativas de calidad se comprendan desde el comienzo.

## CUADRANTE DE TESTING ÁGIL

Importante, la profe habló mucho de esto en la clase.

Los cuadrantes organizan las pruebas según dos dimensiones:

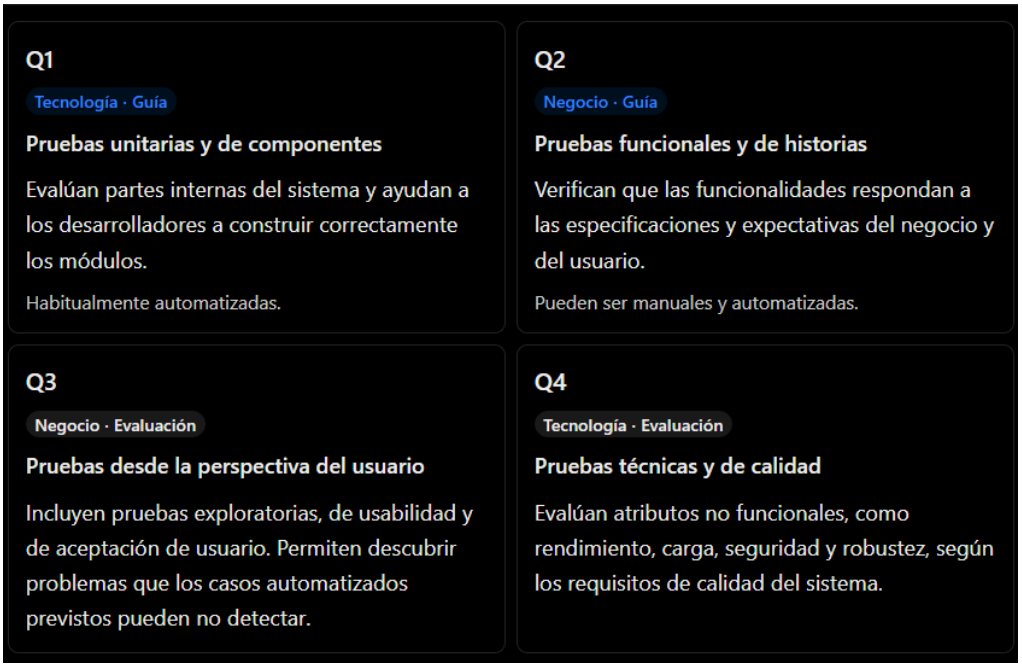
- **Orientación:** pruebas que guían el desarrollo o pruebas que evalúan el producto.
- **Enfoque:** pruebas orientadas al negocio o a la tecnología.

Q1 → Abajo a la izq. Q2 → Arriba a la izq. Q3 → Arriba a la der. Q4 → Abajo a la der.

Los cuatro cuadrantes se usan a lo largo del proyecto.

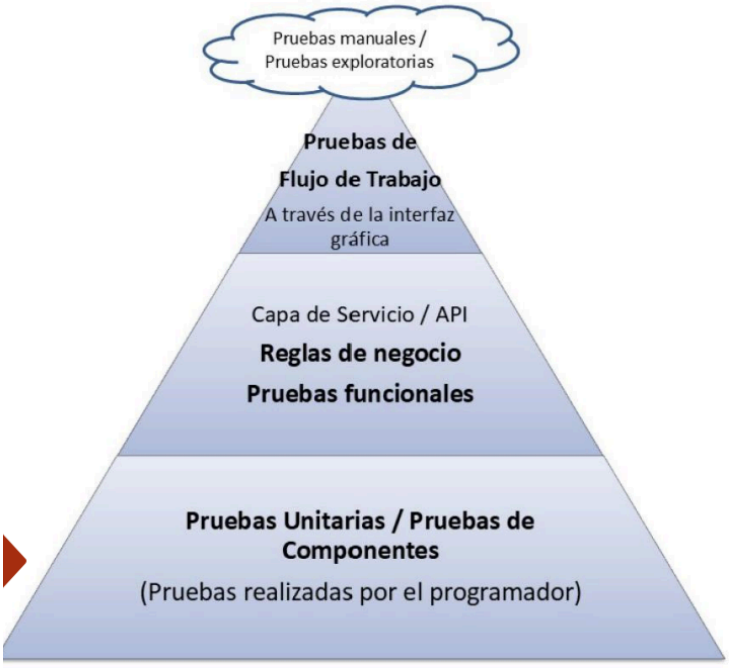
Q1: ¿funcionan correctamente los componentes internos?
Q2: ¿se implementaron las funcionalidades que necesita el negocio?
Q3: ¿cómo se comporta el producto cuando lo evalúa una persona desde la perspectiva del usuario?
Q4: ¿cumple los requisitos técnicos y de calidad?




**Ejemplo con un login:** Q1: test unitario de la función que valida la contraseña. Q2: prueba de aceptación "usuario válido ingresa al sistema". Q3: un tester explora el login probando cosas raras (pegar texto, usar el botón atrás). Q4: probar que soporte 10.000 usuarios simultáneos y que no sea vulnerable a ataques.

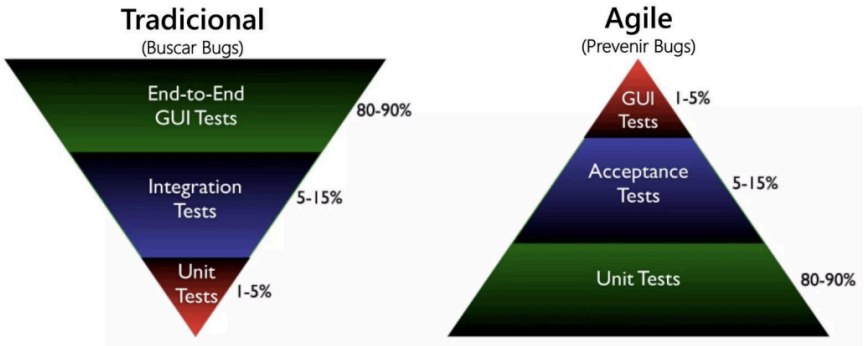
## PIRÁMIDE DE TESTING AGILE




La pirámide representa cómo distribuir el esfuerzo de testing entre distintos niveles. Cuanto más abajo, **más pruebas y más baratas, rápidas y precisas**. Cuanto más arriba, menos pruebas, más lentas, frágiles y costosas.

- **Base** — pruebas unitarias: son numerosas, rápidas y permiten detectar problemas en componentes individuales.
- **Nivel intermedio** — pruebas de integración: verifican que los componentes interactúen correctamente.
- **Parte superior** — pruebas de sistema o de interfaz: evalúan el comportamiento del sistema completo desde una perspectiva más amplia.

*La idea fundamental es que el testing ágil invierte el peso que tradicionalmente se asignaba a las pruebas.*




**Agile:** La base es amplia: más pruebas unitarias, luego de integración y menos pruebas de alto nivel.
**Tradicional:** Se concentra gran parte del esfuerzo en pruebas manuales de alto nivel y queda menos peso para las unitarias.

*La estrategia ágil prioriza detectar los defectos cerca de su origen. Por eso se busca una base amplia de pruebas unitarias automatizadas, acompañada de pruebas de integración y una cantidad menor de pruebas de alto nivel, que suelen ser más costosas y lentas.*

## AUTOMATIZACIÓN DEL TESTING

La automatización es una **premisa básica** del testing ágil: sin ella el **feedback rápido es inviable**, porque no puedo depender del tiempo de las personas. Los 6 beneficios:

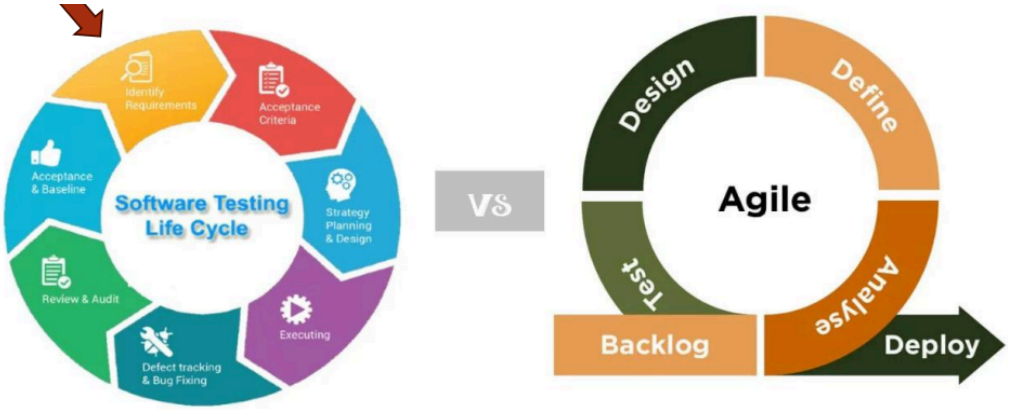
- **Rápida ejecución:** Las pruebas se ejecutan en menos tiempo.
- **Reusabilidad:** las mismas pruebas se ejecutan tantas veces como quiera, sin esfuerzo adicional.
- **Mayor cobertura:** se pueden probar muchos más casos y combinaciones.
- **Mayor precisión:** no hay error humano ni cansancio.
- **Mayor alcance:** permite pruebas imposibles a mano (miles de usuarios simultáneos, miles de combinaciones de datos).
- **Reducción de costos:** a largo plazo, porque requiere una inversión inicial. Disminuye el esfuerzo repetitivo de ejecución y comprobación manual.

La automatización es especialmente importante para las prácticas continuas: si el equipo modifica el software con frecuencia, necesita comprobar rápidamente que los cambios no hayan roto funcionalidades existentes.

*Aclaración:* automatizar no elimina el trabajo humano ni garantiza que se encuentren todos los defectos. Las pruebas exploratorias, la evaluación de usabilidad y el análisis de necesidades siguen siendo importantes. La automatización facilita obtener feedback rápido y repetible.

*Automatizar no significa automatizar todo*. Las pruebas exploratorias y de usabilidad (Q3) siguen siendo manuales.

## TESTING TRADICIONAL FRENTE A TESTING ÁGIL




Se comparan los ciclos de vida tradicional (cascada) vs agile (iterativo/incremental).

Enfoque tradicional:
Requerimientos → Diseño → Desarrollo → Testing

- Las actividades se organizan secuencialmente.
- El testing aparece después del desarrollo.
- El feedback sobre los defectos puede llegar tarde.
- Si el proyecto se retrasa, el tiempo destinado a probar suele reducirse.

Enfoque ágil:
El backlog **alimenta un flujo continuo e iterativo** y el testing es una actividad más del ciclo que entrega valor de forma continua.
Diseño + Desarrollo + Testing → Actividades integradas e iterativas
Backlog → Define → Design → Analyse → Test → Deploy, y se vuelve a empezar.

- El testing se realiza continuamente durante el desarrollo.
- Las pruebas guían las decisiones de implementación.
- El equipo obtiene feedback temprano.
- Se pueden corregir los defectos antes de que se propaguen a otras partes del sistema.

Aclaración: Aunque un ciclo de vida sea iterativo o incremental, el testing puede seguir concentrándose al final de cada iteración. Lo que caracteriza al enfoque ágil es que las pruebas se integran al trabajo de desarrollo, en lugar de dejarse como una actividad posterior.